CRYPTOGRAPHY FOR SECURING CONTROLLER AREA NETWORK (CAN) IN VEHICLES

Nayan Patel (UMID: 27900573)
nayanpat@umich.edu
CIS 505 Project (Item 2), Fall 2021 Term
Faculty: Prof. Thomas Steiner

ABSTRACT

Cars and other vehicles have evolved from mostly all mechanical equipment into the devices controlled by electronic control units (ECUs). The modern vehicles have tens of such ECUs functioning and performing various jobs but for them to function completely, their uninterrupted interconnection is crucial and many times necessary for safety of the driver and passengers and others. [4]

Controller Area Network (CAN) is a primary communication protocol used by these modern vehicle ECUs to exchange information with each other. At the same time when CAN bus provides a robust way for these ECUs to communicate with each other, it also exposes safety-critical functions to security threats. The CAN communication standard does not define any process to secure the messages and hence, it has become a primary target for cyber-attacks. Even though the computer security community has proposed several solutions to alleviate those threats, such solutions have not yet been widely adopted by the automotive industry.

In this paper, the author discusses a CAN security monitoring mechanism that uses a mutually agreed authentication algorithm between receiving and transmitting ECUs.

INTRODUCTION

HOW CAN COMMUNICATION PROTOCOL WORKS?

The Controller Area Network - CAN bus is a message-based protocol designed to allow the Electronic Control Units (ECUs) found in today's automobiles, as well as other devices, to communicate with each other in a reliable, priority-driven fashion. Messages or "frames" are received by all devices in the network, which does not require a host computer. CAN is supported by a rich set of international standards under ISO 11898.

ECUs on a CAN bus are called "nodes." Each node consists of a CPU, CAN controller, and a transceiver, which adapts the signal levels of both data sent and received by the node. All nodes can send and receive data, but not at the same time. All the nodes are synchronized so that they all sample data on the network simultaneously. However, data is not transmitted with clock (time) data, so CAN is not truly a synchronous bus. [7]

With CAN, all data are sent in frames, and there are four types:

- **Data frames** transfer data to one or many receiver nodes
- **Remote frames** ask for data from other nodes
- **Error frames** report errors
- **Overload frames** report overload conditions

There are two variants of the message length: standard and extended. The real difference is the additional 18-bit identifier in the arbitration field. Figure 2 provides details of both the message variants.

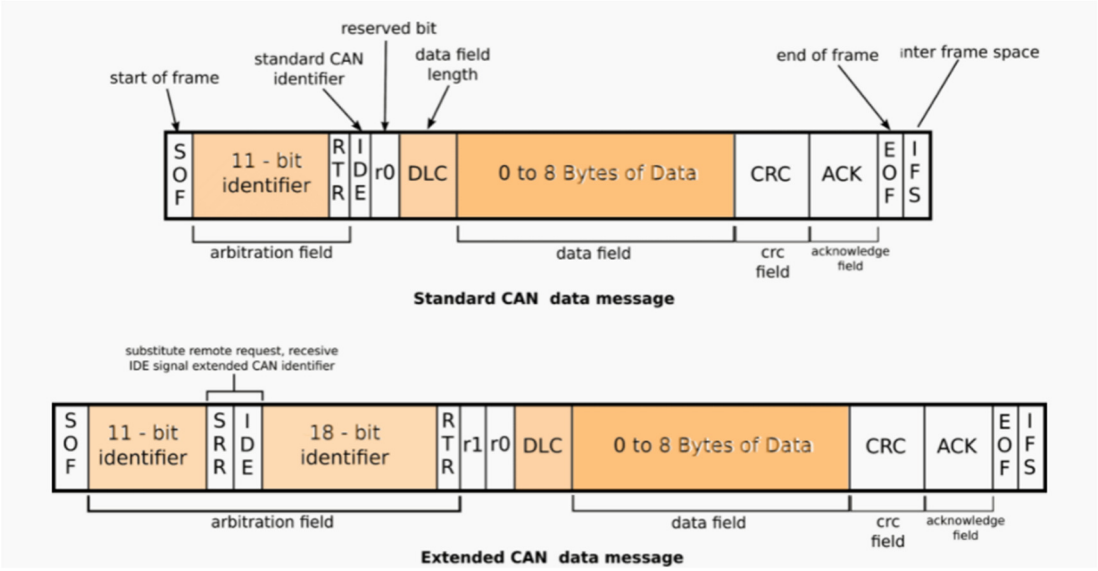

Figure 1: Classic CAN (Standard and Extended) data frame format

Types of CAN Bus:

The ISO 11898 standard defines two major types of CAN bus: ***Classic CAN and CAN-FD***. The main difference between these two types is the actual length of the data field. The Classic CAN frame carries anywhere between 0 to 8 bytes of data whereas CAN-FD carries anywhere between 0 to 64 bytes of data. Besides, the data length, Classic and CAN-FD have another major difference which is the transmission rate. The Classic CAN transfer rate can be anywhere between 125 kbps (Low Speed CAN) to 1Mbps (High Speed CAN). The CAN-FD bus can be operated at very fast rate of 1Mbps to 8Mbps.

The data frames shown in Figure 1 is Classic CAN frame whereas Figure 2 depicts the CAN-FD data frame format. [10]

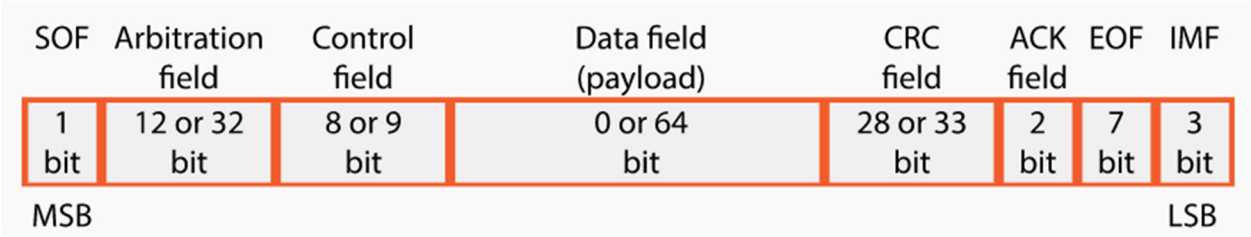

Figure 2: CAN-FD data frame format

SECURITY THREAT IN CAN COMMUNICATION

When the CAN protocol was first developed and released in 1986 by Robert Bosch GmbH, security requirements were not considered as it was designed for a use in closed environment. However, the connection of the in-vehicle network to external network through various illustrated in Figure 3 brings security risks. [5]

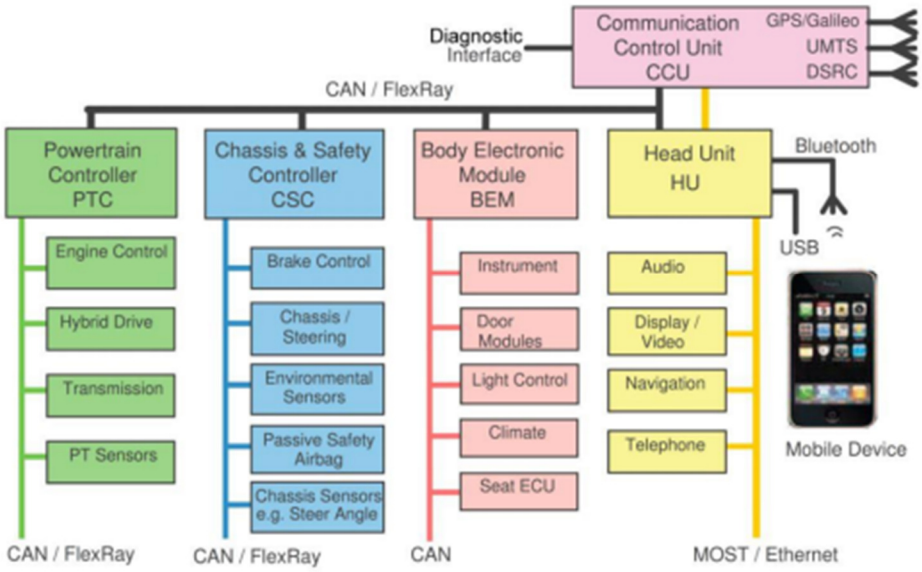

Figure 3: Typical In-vehicle Network

Following use cases represent possible spoofing and replaying techniques used to hack into and manipulate vehicle CAN network to take over the control from one or more ECUs:

Use case 1: Unauthorized alteration of ECU software: Figure 4 shows an example of spoofed or replayed message transmission by an ECU after its authorized program is replaced by a malicious program. [3]

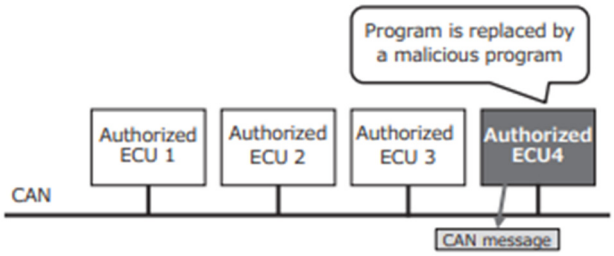

Figure 4: Replacement of authorized ECU program by malicious program

Use case 2: Connection of unauthorized device: Figure 2 shows an example of spoofed or replayed message transmission by an unauthorized device connected on a CAN bus.

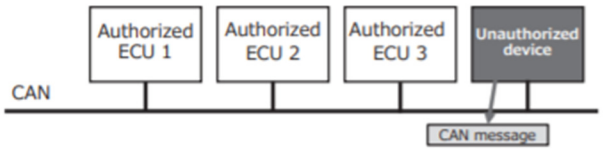

Figure 5: Connection of unauthorized device on CAN bus

The CAN protocol is vulnerable to spoofing and replaying attacks since it has no authentication system as described before.

DEFENSE MECHANISMS TO PROTECT CAN BUS

The history of attacks on CAN bus clearly show that the protocol is very vulnerable and requires cyber defense mechanisms for safe driving. Following are different mechanisms to solve the problems of CAN bus cyber-attacks. [4]

Method	Pros	Cons
Network Segmentation	Limit access to end-user	Increased cost, difficult to maintain
Encryption	Hardened attacks, confidential data transmission	Increased computational power, increased traffic, weak encryption due to frame width limitation
Authentication	Secure data transmission	Increased computational power, increased traffic
Intrusion Detection	Detect anomalies and attacks	Complicated algorithm design, cannot guarantee all kind of attacks

Table 1: Methods to secure the CAN bus

In this paper, an Authentication technique is presented that provides strong security against the spoofing attacks. In particular, a monitoring node authenticates each node and verifies the message authentication codes assigned to the CAN messages. In the following sections, the rationale behind choosing the authentication technique and the details of how that is implemented is discussed.

CONCEPT OF CAN MESSAGE AUTHENTICATION

Authentication method discussed here uses Message Authentication Code (MAC). The objective of the discussed security monitoring system is to make the vehicle system's CAN bus nodes prepared against the spoofing attacks.

A transmission node (an authorized ECU) calculates the MAC using an encryption key that is exchanged or stored in the key slots in Hardware Security Module (HSM) or Security Hardware Extension (SHE). Part of this generated MAC is embedded inside the CAN message payload and the data frame is transmitted on the CAN bus. The receiving node (i.e., monitoring node), upon receipt of the data frame, extracts the MAC value and verifies it against the same encryption key as was used by the transmission node. If the CAN controller inside the ECU detects the MAC value missing or wrong, it will reject the message and set appropriate diagnostics. [3]

Figure 6 depicts high level overview of the concept defined above.

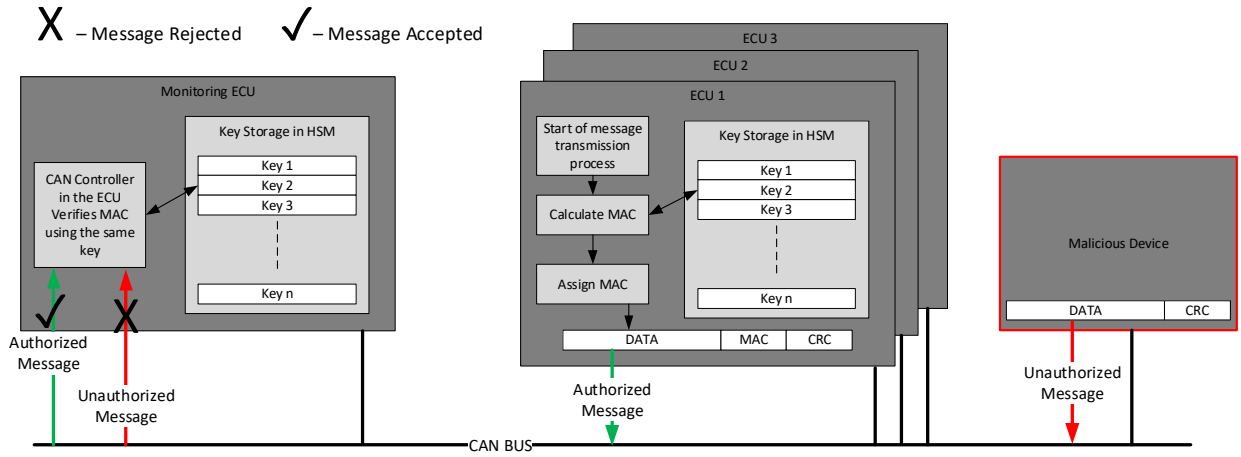

Figure 6: MAC for CAN Bus Security

The entire process of CAN message authentication is divided into two main phases:

1) Key distribution or exchange
2) Authentication using MAC calculation

1) KEY DISTRIBUTION OR EXCHANGE

There are two ways the key can be distributed or exchanged between transmitting and receiving nodes:

ONE-TIME KEY DISTRIBUTION DURING LIFE CYCLE OF THE VEHICLE

If all the ECUs in the vehicle which are taking part of CAN Message Authentication technique have either the SHE or HSM to securely store the keys, it would be very efficient to distribute a predefined set of keys and use them based on mutually agreed manner to calculate the MAC value while transmitting a message and to verify MAC value of a received message. Figure 6 above shows this specific way where the keys are stored in HSM and both Tx and Rx nodes use the same key to generate and verify the MAC value embedded in the CAN data frame.

This way is very less intense for computational power and bus load as the keys are exchanged only one time in the entire life cycle of the vehicle. But at the same time, this requires some specialized infrastructure at the ECU manufacturing or vehicle assembly as they need to be equipped with secured distribution of set of keys in each ECU during the manufacturing time itself.

KEY EXCHANGE AT EACH IGNITION CYCLE

In this specific way, a new key is received from Cloud or generated by one master node (most times it is Telematic Control Unit) and provided to each ECU so that all nodes have an identical key. Since this key is used by the ECUs for CAN message authentication, the distribution process requires a strong secured mechanism.

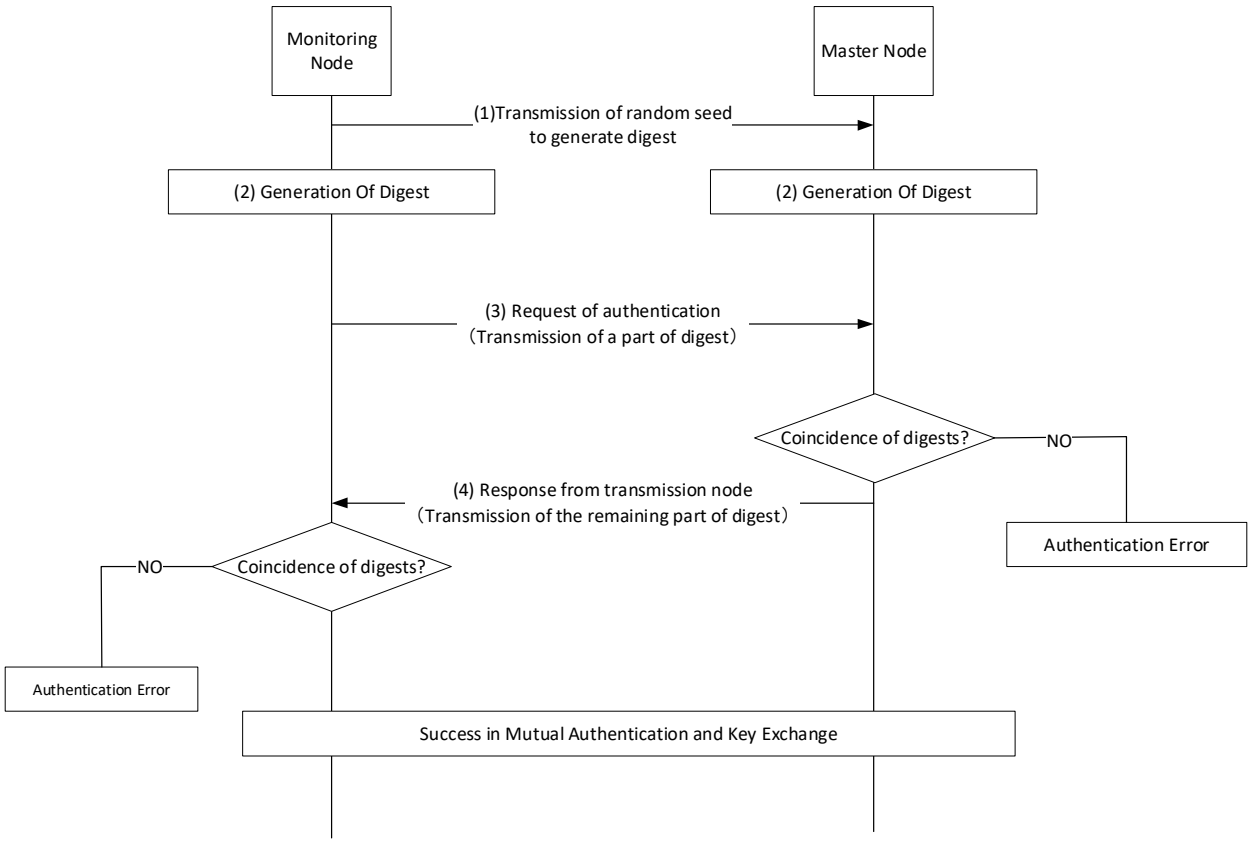

Figure 7: Key Exchange at Ignition Cycle

The entire process of key exchange at ignition cycle is described in the following steps. Also, the Figure 7 above depicts the flow of the same.

- The monitoring node transmits a random seed to the transmission node.
- After the transmission node receives the random seed from the monitoring node, both nodes calculate the digest. A hash function is usually used for the digest calculation.
- The monitoring node transmits a part of the digest to the transmission node.
- The transmission node calculates the digest and compares it with the part of the digest transmitted from the monitoring node. When the two digests are equal to each other, the transmission node transmits the subsequent digests to the monitoring node.
- The monitoring node conducts authentication by calculating the digest and comparing it with the part of the digest received from the transmission node.

2) AUTHENTICATION USING MAC CALCULATION

There are two different algorithms that are feasible in the ECU embedded systems in the vehicles for calculating the MAC value:

1. Cipher-Based Message Authentication (CMAC)
2. Hash-Based Message Authentication (HMAC)

Generally, CMAC is slower than HMAC, as hashing methods are generally faster than block cipher methods. In most cases HMAC will work best, but CMAC works better in the embedded hardware where there is hardware accelerator for block cipher is available.

CMAC method is used in the paper for CAN message authentication as modern-day vehicle ECUs are using the microcontrollers (µC) or system on chips (SOCs) which have in-built HSM or SHE cores which normally have the AES-128 hardware accelerator.

CMAC ALGORITHM FOR CAN MESSAGE AUTHENTICATION

The specification of CMAC algorithm (rfc4493) defines two important major steps in calculating CMAC value:

1) Subkey Generation Algorithm
2) CMAC Calculation Algorithm

1) SUBKEY GENERATION ALGORITHM

[8] The subkey generation algorithm, takes a secret key K, which is just the key for AES-128.

The outputs of the subkey generation algorithm are two subkeys, K1 and K2.

We write (K1,K2) := *Generate_Subkey(K)*

Subkeys K1 and K2 are used in both MAC generation and MAC verification algorithms. K1 is used for the case where the length of the last block is equal to the block length. K2 is used for the case where the length of the last block is less than the block length.

Prerequisites:

Constants: *const_Zero* is 0x00000000000000000000000000000000

const_Rb is 0x00000000000000000000000000000087

Variables: *L* for output of AES-128 applied to 0^128

Input:

K (128-bit key)

Output:

K1 (128-bit first subkey)

K2 (128-bit second subkey)

Steps:

1. L := AES-128 (*K, const_Zero*);
2. if MSB(*L*) is equal to 0

   then *K1 := L* << 1;

   else *K1* := (*L* << 1) XOR *const_Rb*;
3. if MSB(*K1*) is equal to 0

   then *K2* := *K1* << 1;

   else *K2* := (*K1* << 1) XOR *const_Rb*;
4. return *K1*, *K2*;

2) CMAC CALCULATION ALGORITHM

Following sections describe the CMAC calculation algorithms for generating a CMAC value from one or more block of data and verifying a CMAC by a receiving node. [1]

CMAC GENERATION

The Algorithm of CMAC works following way per the specification of MAC generation process of CMAC

We write (*T*) := *Generate_CMAC(b, K, Tlen, M)*

Prerequisites:

Block cipher with block size b;

Symmetric Key K:

MAC length parameter *Tlen*

Input:

Message M of bit length *Mlen*

Output:

MAC *T* of bit length *Tlen*

Steps:

1. Fetch one key Ki to Kn from the HSM based on pre-defined/agreed sequence by all the nodes to produce subkeys *K1* and *K2*.
2. If *Mlen* = 0, let *n* = 1; else, let n = *Mlen*/*b*.
3. Let *M1, M2, ... , Mn-1, Mn* * denote the unique sequence of bit strings such that *M = M1 || M2 || ... || Mn-1 || Mn*, where *M1, M2,..., Mn-1* are complete blocks
4. If *Mn* is a complete block, let $Mn = K1 \oplus Mn$; else, let $Mn = K2 \oplus (Mn \,||\, 10^j)$, where $j = nb - Mlen - 1$.
5. Let $C_0 = 0^b$.
6. For *i = 1 to n*, let $C_i = CIPH_K(C_i\text{-}1 \oplus M_i)$.
7. Let *T = MSBTlen(Cn).*
8. Return *T*.

In Step 1, the subkeys are generated from the key. In Steps 2–4, the input message is formatted into a sequence of complete blocks in which the final block has been masked by a subkey. There are two cases (Figure 8):

- If the message length is a positive multiple of the block size, then the message is partitioned into complete blocks. The final block is masked with the first subkey; in other words, the final block in the partition is replaced with the exclusive-OR of the final block with the first subkey. The resulting sequence of blocks is the formatted message.
- If the message length is not a positive multiple of the block size, then the message is partitioned into complete blocks to the greatest extent possible, i.e., into a sequence of complete blocks followed by a final bit string whose length is less than the block size. A padding string is appended to this final bit string, in particular, a single '1' bit followed by the minimum number of '0' bits, possibly none, that are necessary to form a complete block. The complete final block is masked, as described in the previous bullet, with the second subkey. The resulting sequence of blocks is the formatted message.

In Steps 5 and 6, the cipher block chaining (CBC) technique, with the zero block as the initialization vector, is applied to the formatted message. In Steps 7 and 8, the final CBC output block is truncated according to the MAC length parameter that is associated with the key, and the result is returned as the MAC.

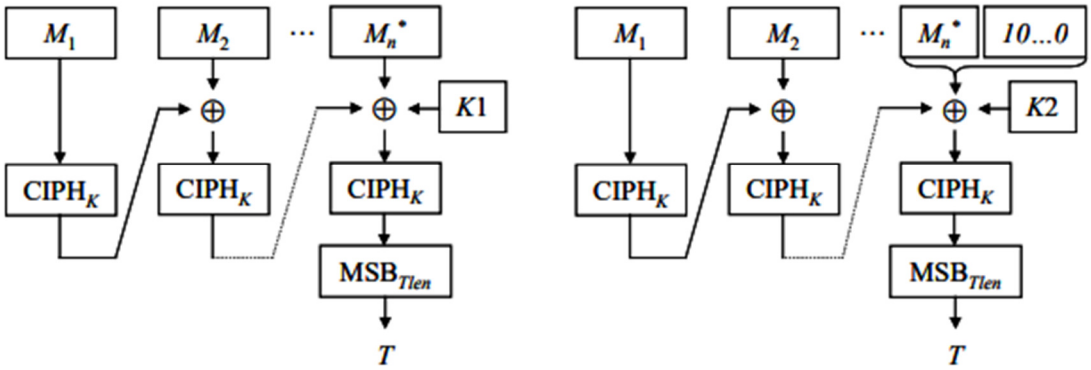

Figure 8: Illustration of the two cases of CMAC generation

CMAC VERIFICATION

The following is a specification of the MAC verification process of CMAC:

We write (*bool*) := *Verify_CMAC(b, K, Tlen, M, T')*

Prerequisites:

block cipher CIPH with block size *b*;

key *K*;

MAC length *Tlen*.

Input:

message *M* of bit length *Mlen*;

received *MAC T'*.

Output:

VALID or *INVALID*.

Steps:

1. Apply the MAC generation process in CMAC GENERATION section to M to produce T.
2. If T = T', return VALID; else, return INVALID.

In Step 1, the MAC generation process in defined in CMAC GENERATION section is applied to the message, and, in Step 2, the resulting MAC is compared with the received MAC to determine its validity.

USE OF CMAC IN CAN MESSAGE AUTHENTICATION

The CMAC is given to a part of the payload to assure the integrity of the CAN message, as shown in Figure 9. Importantly, the CAN bus has a limitation of frame length, so only a part (only some bits) of 128 bit CMAC result is inserted in the frame.

The role of the CMAC is to protect data from spoofing and replaying on the in-vehicle network. For the purpose of maintaining interchangeability with existing CAN controllers, the system does not replace CRC with the CMAC. When the monitoring node receives a message, the node checks the CAN-ID to judge whether or not the CMAC has been given to the message. When the CMAC has been given to the message, the monitoring node immediately calculates the CMAC to verify the CMAC. At this point of time, no node has transmitted an ACK signal to the CAN bus. However, if the monitoring node detects the CMAC error, the node overwrites the spoofed or replayed message with the error frame up to the end of frame (EOF) or rejects the message completely without further action.

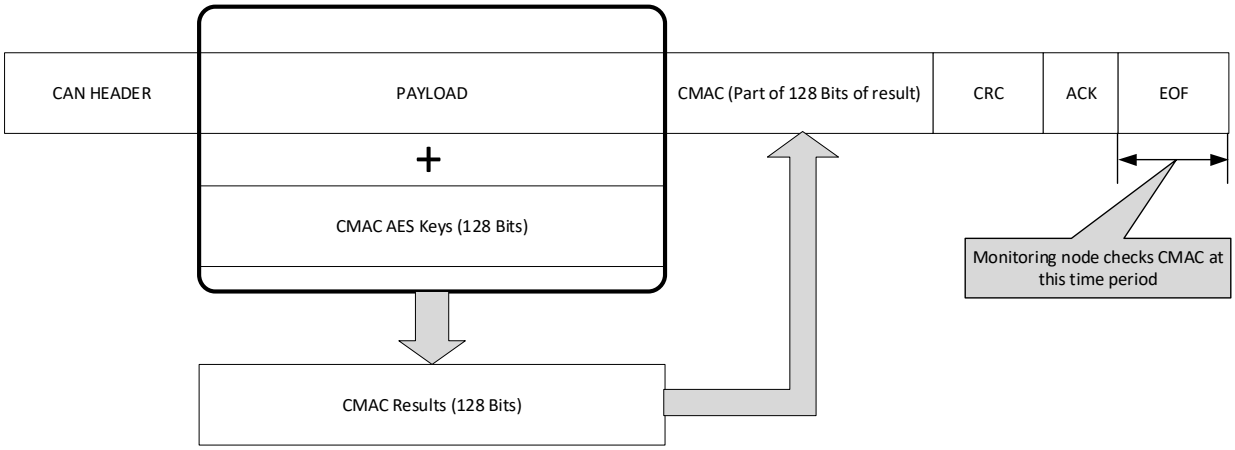

Figure 9: CMAC Insertion and calculation Timing

PERFORMANCE ANALYSIS OF CMAC AUTHENTICATION IN CAN MESSAGE

SECURITY OF THE MESSAGES

Since the maximum payload of CAN bus is 8 bytes, it is impossible to accommodate the complete CMAC value in a single frame. Hence, let's assume we use only 1 byte (8 bits) out of total 128 bits of CMAC to the frames.

For CMAC of 1 byte in length, the total number of necessary attacks on a particular message is 2^8 = 256. Hence, we can say that when total 256 messages are transmitted, one possibility of spoofing can occur successfully. Though this number is not enough to perfectly guarantee the security of the CAN, rotating the keys after transmission of several messages and other additional preventive measures will make it more difficult to attack the CAN by spoofing.

REAL-TIME LIMITATION

As the number of nodes increases by one, the number of messages necessary for authentication increases by the same number. Assuming the CMAC length used in the authenticated frame is 1 byte, the 4 bits length of the counter for preventing replay has to be added which altogether adds an overhead of 12 bits. This overhead accounts for 19% of the total payloads and this overhead can limit the bandwidth of the CAN bus for all the nodes.

Also, the calculating the CMAC value requires significant amount of CPU load and precise time for completion of job. Therefore, in the ECUs with less processing power have difficulties to implement this.

CONCLUSION

Attacks on in-vehicle control units via in-vehicle control networks have been reported recently and various measures for protecting the networks from attacks have been discussed. However, most of these measures must be implemented for every node. In this paper, we discussed an authentication technique using CMAC and discussed its demonstration results. This authentication technique when provides good security from spoofing and replaying malicious messages, it brings some challenges at the same time but with increased resources and more CPU power, those challenges are overcome.

There is significant scope of improvement still possible in this area. Many researchers are working towards different techniques to minimize the overhead on payload and CPU load without sacrificing the security performance but it's extremely hard to balance the tradeoff between them. [6]

BIBLIOGRAPHY

[1] Dworkin), N. (. (n.d.). *Recommendation for Block Cipher.* Retrieved from https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-38B.pdf

[2] Garcia, A.-I. R. (n.d.). *LeiA: A Lightweight Authentication Protocol for*. Retrieved from https://www.cs.bham.ac.uk/: https://www.cs.bham.ac.uk/~garciaf/publications/leia.pdf

[3] Hiroshi UEDA, R. K. (n.d.). *Security Authentication System for In-Vehicle Network*. Retrieved from https://global-sei.com/technology/tr/bn81/pdf/81-01.pdf

[4] Mehmet Bozdal, M. S. (2020). Evaluation of CAN Bus Security Challenges . *A Survey on CAN Bus Protocol: Attacks, Challenges, and Potential Solutions.* Southend, UK: In Proceedings of the 2018 International Conference on Computing, Electronics & Communications Engineering (iCCECE).

[5] MERTOL, S. (n.d.). *Thesis on SECURE MESSAGE AUTHENTICATION PROTOCOL FOR CAN.* Retrieved from https://etd.lib.metu.edu.tr/upload/12625136/index.pdf

[6] Ryo Kurachi, H. T. (n.d.). Towards Minimizing MAC Utilization. *AutoSec '20, March 18, 2020, New Orleans, LA, USA.* Retrieved from https://dl.acm.org/: https://dl.acm.org/doi/pdf/10.1145/3375706.3380557

[7] Smith, G. M. (n.d.). *What Is CAN Bus (Controller Area Network) and How It Compares to Other Vehicle Bus Networks*. Retrieved from https://dewesoft.com/: https://dewesoft.com/daq/what-is-can-bus

[8] Society, T. I. (n.d.). *The AES-CMAC Algorithm*. Retrieved from ietf.org: https://datatracker.ietf.org/doc/html/rfc4493.html

[9] Soja, R. (n.d.). *Automotive Security: From Standards to*. Retrieved from avnet.com: https://www.avnet.com/wps/wcm/connect/onesite/31385b8d-a332-4356-8089-644b9a04b813/automotive-security-from-standards-to-implementation-white-paper.pdf?MOD=AJPERES&CVID=lID2Dt5&id=1491008660934

[10] Wikipedia. (n.d.). *CAN bus*. Retrieved from https://en.wikipedia.org/: https://en.wikipedia.org/wiki/CAN_bus

[11] Zeinab El-Rewini, K. S. (n.d.). Cybersecurity challenges in vehicular communications. *https://www.osti.gov/servlets/purl/1641749*.